易宿酒店管理平台

全栈工程项目文档

作　　者：G,C,R
学　　校：(请填写学校名称)
指导教师：(请填写指导教师姓名)
版　　本：1.0
日　　期：2026 年 02 月 13 日

目录

1 项目概述 ... 3
1.1 项目背景与意义 ... 3
1.2 系统定位与三端架构 ... 3
1.3 技术选型理由 ... 3
1.4 项目亮点与创新点 ... 3
2 需求分析 ... 5
2.1 用户角色与权限矩阵 ... 5
2.2 功能需求 ... 5
2.3 非功能需求 ... 6
3 系统设计 ... 7
3.1 总体架构 ... 7
3.2 技术栈版本清单 ... 7
3.3 项目文件结构 ... 8
3.4 模块划分 ... 10
3.5 数据库设计 ... 11
3.6 核心业务流程 ... 15
4 具体实现 ... 18
4.1 认证与鉴权实现 ... 18
4.2 预订事务实现 ... 18
4.3 AI 推荐实现 ... 19
4.4 前端 HTTP 客户端实现 ... 19
4.5 商户数据看板实现 ... 20
4.6 地图导航实现 ... 20
4.7 深色模式实现 ... 20
4.8 小程序首页搜索实现 ... 21
4.9 酒店列表与筛选实现 ... 21
4.10 酒店详情页实现 ... 22
4.11 管理员酒店审核实现 ... 22
5 接口文档 ... 23
5.1 接口约定 ... 23
5.2 认证接口 `/api/auth` ... 23
5.3 用户接口 `/api/users` ... 23
5.4 酒店接口 `/api/hotels` ... 23
5.5 预订接口 `/api/bookings` ... 24
5.6 房型与库存接口 `/api/room-types` ... 24
5.7 评价接口 `/api/reviews` ... 25
5.8 收藏接口 `/api/favorites` ... 25
5.9 AI 推荐接口 `POST /api/ai/recommend` ... 25
5.10 文件上传接口 `POST /api/upload` ... 26
6 部署与运行 ... 27
6.1 环境依赖 ... 27
6.2 环境变量配置 ... 27
6.3 种子数据与测试账号 ... 27
6.4 后端 + Web 启动 ... 27
6.5 小程序端启动 ... 28
6.6 常见问题排查 ... 28

1 项目概述

1.1 项目背景与意义

随着在线旅游市场的持续扩张，酒店行业的数字化管理需求日益迫切。现有平台普遍存在以下痛点：商户端功能薄弱，无法进行动态定价；管理员审核流程不透明，缺乏操作追溯；用户端体验割裂，自然语言搜索能力有限。

易宿（YiSu）针对上述痛点，构建了一套覆盖「平台管理—商户运营—用户预订」完整链路的全栈酒店管理系统。系统核心亮点如下：

- **AI 智能推荐**：接入大语言模型，用户以自然语言描述需求（如"找北京性价比高、带早餐的酒店"），系统自动提取结构化搜索条件，结合实时库存数据流式推荐；
- **事务性预订**：通过 `prisma.$transaction` 对每个入住日期逐日 upsert 库存并原子累加 `booked` 计数，从数据库层面杜绝超卖；
- **精细化权限**：三角色（USER / MERCHANT / ADMIN）+ 细粒度权限（HOTEL_AUDIT / HOTEL_DELETE）双层鉴权，每次状态变更均写入审计日志；
- **多端统一**：Web 后台（Next.js）与微信小程序（Taro）共用同一套 REST API，业务逻辑零重复。

1.2 系统定位与三端架构

系统按角色划分为三个端，相互独立但共用同一套后端 API：

端	载体	核心职责
管理员后台	Web 浏览器	酒店审核审批、上下线管理、城市与标签维护、审计日志查询
商户后台	Web 浏览器	酒店与房型 CRUD、动态定价日历、预订状态流转、数据可视化看板
用户端	微信小程序	关键词与城市搜索、在线预订、地图导航、AI 对话推荐、评价与收藏

三端共用同一套 Next.js API Routes 后端，前端通过 HTTP Bearer Token 认证调用，数据库统一使用 MySQL 8.x + Prisma ORM。这种架构的优势在于：一次接口变更即可同步生效于所有端，无需维护多套后端服务。

1.3 技术选型理由

技术	版本	选型理由
Next.js	16.1.6	App Router 支持服务端渲染，API Routes 合并部署无需独立后端，减少运维复杂度
TypeScript	5.x	Prisma 自动生成数据库类型，前后端共享类型定义，全链路类型安全
Prisma ORM	6.19.2	声明式 schema 驱动，`$transaction` 原子操作，内置迁移工具，与 TypeScript 深度集成
MySQL	8.x	成熟关系型数据库，支持事务，适合预订类强一致性场景
Ant Design	6.2.2	企业级组件库，Table/Drawer/Modal/Form 开箱即用，适合中后台页面
Recharts	3.7.0	基于 SVG 的 React 图表库，折线图/柱状图/饼图直接声明式使用
Taro	4.1.11	一套 React 代码编译微信/支付宝/抖音/H5 等 8+ 平台，降低多端维护成本
LangChain	1.2.18	封装 LLM 调用链，`withStructuredOutput` 实现意图提取，支持流式输出与多模型切换
JWT	jsonwebtoken 9.0.3	无状态鉴权，双 Token 机制（Access 1h + Refresh 7d），适合前后端分离架构
Zod	4.3.6	运行时 Schema 校验与 TypeScript 类型推导无缝结合，与 LangChain 结构化输出复用同一套定义

1.4 项目亮点与创新点

本项目在技术选型与工程实践上有以下几个核心亮点，区别于同类课程设计项目：

① **事务性防超卖机制**：预订接口通过 `prisma.$transaction` 对每个入住日期逐日 upsert RoomAvailability，在数据库行锁保护下原子递增 `booked` 计数，从存储层根本杜绝超卖，无需应用层加锁或队列。

② **大语言模型两阶段推荐**：将 LLM 调用分为「确定性意图提取（temperature=0）」和「多样性文案生成（temperature=0.7）」两阶段，前者用 Zod Schema 约束结构化输出（目的地/价格区间/关键词），后者注入真实库存数据生成流式推荐；两者职责分离、互不干扰。

③ **精细化 RBAC 双层鉴权**：JWT Token 携带角色完成粗粒度校验，数据库 Role→Permission 关联完成细粒度校验（HOTEL_AUDIT / HOTEL_DELETE），每次状态变更原子写入审计日志，满足可追溯性要求。

④ **多端统一架构**：管理员 Web、商户 Web、用户小程序三端复用同一套 Next.js API Routes，通过 Taro 实现"一套 React 代码多端编译"，接口变更一次生效于全平台。

⑤ **全栈类型安全**：Prisma 自动生成数据库操作类型，前后端共享 TypeScript 类型定义（`app/types/index.ts`），LangChain 与 Zod 共用同一 Schema，全链路无运行时类型错误。

⑥ **工程化规范**：接口通过 JSDoc 注释自动生成 OpenAPI 3.0 Swagger 文档（`/api-doc`）；文件上传实现文件名安全化防路径穿越；Prisma Client 通过单例模式防止 Next.js 热更新时的连接泄漏。

2 需求分析

2.1 用户角色与权限矩阵

系统设计三种角色，通过数据库 `Role` 与 `Permission` 表实现 RBAC（基于角色的访问控制）。角色信息在 JWT Payload 中携带，API 层由 `verifyAuth()` 解码后按需进行二次权限查询。

功能模块	普通用户 (USER)	商户 (MERCHANT)	管理员 (ADMIN)
酒店搜索与浏览	✓	✓	✓
在线预订	✓	×	×
查看 / 取消本人订单	✓	×	×
提交评价（完成订单后）	✓	×	×
收藏酒店	✓	×	×
酒店 CRUD（本人）	×	✓	✓（全部）
房型与动态定价管理	×	✓	×
查看本酒店预订	×	✓	✓（全部）
数据可视化看板	×	✓	×
酒店审核（通过 / 拒绝）	×	×	✓（需 HOTEL_AUDIT 权限）
酒店上下线	×	商户可下线本人	✓
城市与标签管理	×	×	✓
审计日志查询	×	×	✓
删除任意酒店	×	×	✓（需 HOTEL_DELETE 权限）

权限判断在 API 层通过 `verifyAuth()` + 查询 `role.rolePermission` 实现双层校验，前端通过 `useAuth(requiredRole)` Hook 进行页面级路由保护（角色不匹配时强制跳转首页）。

2.2 功能需求

2.2.1 用户端（微信小程序）

微信小程序是面向 C 端用户的核心交互入口。用户完成注册登录后，可通过以下核心流程完成「发现—预订—评价」的完整旅行消费链路：

功能	主要流程	关键输出
酒店搜索	首页选城市 + 输入关键词 + 日历选日期 + 价格/星级筛选 → 跳转列表页	匹配酒店列表（含各房型最低价、剩余间数）
在线预订	详情页选房型 → 日历选日期（实时算价）→ BookingConfirm 填写入住人 → 提交	创建 Booking 记录（status=pending）
订单管理	orderList 查看状态 → orderDetail 查看详情 → 待确认订单可取消	订单状态更新
地图导航	详情页点击“地图” → hotelMap 页显示用户蓝点 + 酒店标记 → 点击“导航”	调起微信 `wx.openLocation` 内置导航
AI 助手	任意页面点击悬浮按钮 → 输入自然语言 → 流式接收推荐	推荐酒店 + 流式文字说明
评价	已完成订单 → reviewList 提交 1–5 星 + 文字 → 查看 / 删除	Review 记录（一单一评，防重复）
收藏	详情页心形按钮 toggle → favoriteList 管理	Favorite 记录（复合主键防重复）
深色模式	系统主题跟随或 mine 页手动切换 → `Taro.eventCenter` 广播全局 CSS 变量	全页面无闪烁主题切换

2.2.2 商户后台（Web）

商户后台是 B 端运营的核心工具，为酒店经营者提供从「入驻—配置—运营」的全生命周期管理能力：

功能	主要流程	关键输出
数据看板	Dashboard 自动加载商户全量预订，前端聚合计算	今日预订/月营收/入住率 + 折线图/柱状图/饼图
酒店管理	hotel 页面 CRUD + 多图上传（POST /api/upload）+ 城市/标签选择 → 提交审核	Hotel 记录（status=pending，等待管理员审核）
房型管理	hotels 页面下属房型 CRUD → 设置名称/床型/价格/库存/折扣/设施/图片	RoomType 记录
动态定价	calendar 日历视图选日期范围 → 批量设置价格/配额/关闭房态	RoomAvailability 批量 upsert
预订管理	bookings 页面查看本酒店预订 → 按状态筛选 → 操作状态流转（确认/拒绝）	Booking.status 更新

2.2.3 管理员后台（Web）

管理员后台承担平台层面的内容治理职责，重点在审核合规与数据可信度：

功能	主要流程	关键输出
酒店审核	review 页面查看 pending 酒店 → Drawer 展示详情（图片/设施/房型）→ 通过或拒绝（填原因）	status 变更 + HotelAuditLog 原子写入（事务）
上下线管理	hotels 页面对 published 酒店操作下线/上线	status 在 offline ↔ published 间切换
城市管理	locations 页面 CRUD 城市记录	Location 增删改，供酒店归属地选择
标签管理	tags 页面 CRUD 酒店标签	Tag 增删改，供酒店多对多关联筛选
审计日志	hotels/[id]/audit_log 查询历史变更	HotelAuditLog 列表（含操作人/时间）

2.3 非功能需求

系统在功能需求之外，对可靠性、安全性和可维护性提出了明确约束：

类型	具体说明
并发安全	`prisma.$transaction` 事务内逐日 upsert `RoomAvailability`，`booked >= quota` 时抛错整体回滚，从数据库层面防止超卖。并发预订时同一天的 `booked` 值由 MySQL InnoDB 行锁保护原子递增
安全性	密码 `bcryptjs` 哈希（salt rounds=10）存储，明文不落库；Access Token 1h + Refresh Token 7d 双 Token 机制；`verifyAuth()` 在每个受保护接口头部校验 JWT；状态变更/删除额外检查 `Permission.name`（HOTEL_AUDIT / HOTEL_DELETE）
数据一致性	`RoomAvailability` 唯一约束 `(roomTypeId, date)` + `upsert` 幂等写入；`Review.bookingId` 唯一索引（一单一评）；`Favorite` 复合主键 `(userId, hotelId)` 防重复收藏；`HotelAuditLog` 在状态变更事务内原子写入，保证日志不漏不重
可观测性	`HotelAuditLog` 记录 operatorId / oldStatus / newStatus / comment / createdAt，完整追溯审核历史；Next.js 开发模式下请求日志实时打印到 stdout
跨平台	Taro 4 编译为微信/支付宝/抖音/H5 等平台；后端 Next.js API Routes 无状态，可水平扩展；Swagger UI 在 `/api-doc` 自动生成接口文档
可扩展性	Prisma schema 驱动变更，新增表只需修改 schema + `prisma db push`；Service 层隔离前端与后端，接口变化对组件透明；LangChain 模型实例可通过 `.env` 切换不同大模型供应商（OpenAI / DouBao / Kimi）

3 系统设计

3.1 总体架构

系统采用前后端合一的五层架构，Next.js 同时承担 SSR 前端渲染与 REST API 的双重职责。各层职责如下：

- **展示层**：Web 侧由 Next.js App Router Page 直接渲染；小程序侧由 Taro 组件编译为微信原生组件；
- **服务层**：对 API 调用进行封装，Web 侧用 Axios（`app/lib/request.ts`），小程序侧用 `Taro.request`；
- **接口层**：Next.js API Routes 处理 HTTP 请求，`verifyAuth()` 统一校验 JWT，细粒度权限查询 Permission 表；
- **数据访问层**：Prisma Client 单例（`app/lib/prisma.ts`）提供类型安全的 ORM 操作，支持 `$transaction`；
- **存储层**：MySQL 8.x，InnoDB 引擎保证行锁与事务 ACID。

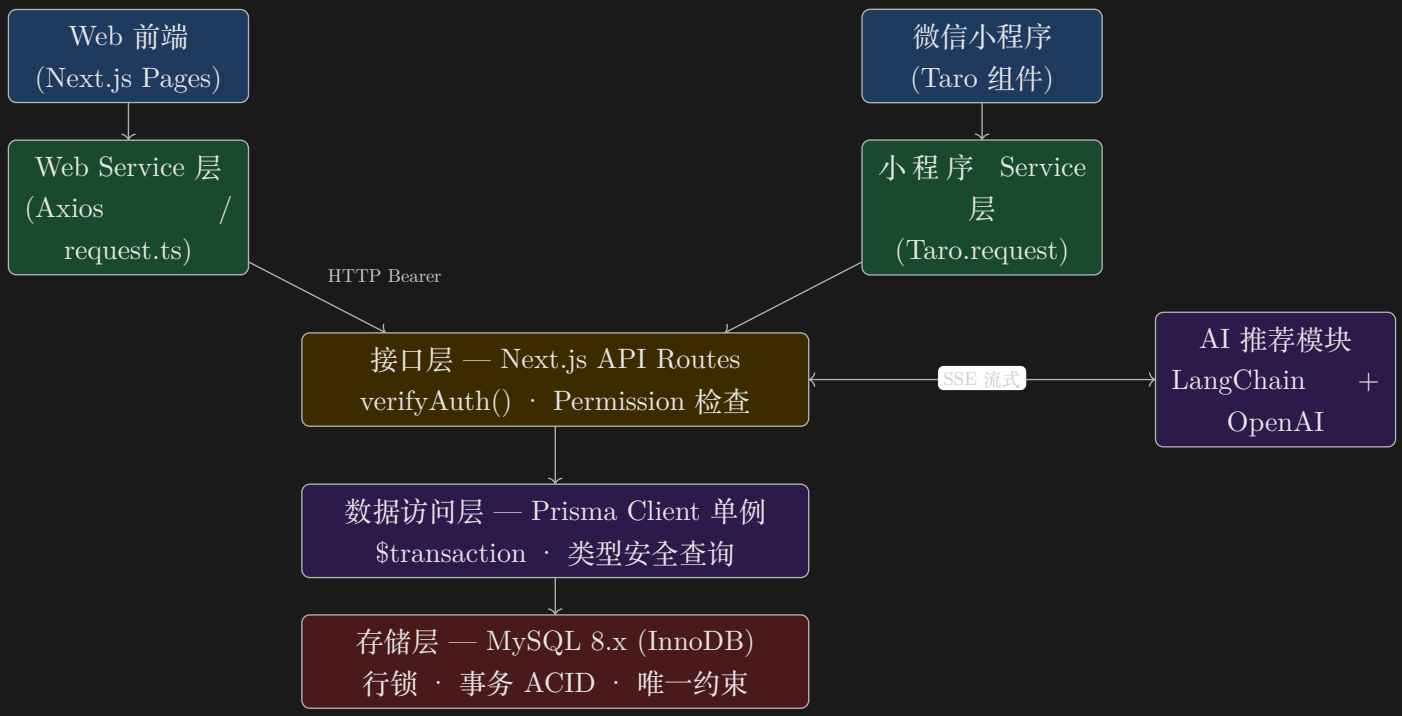

Figure 1: 易宿系统总体五层架构

AI 推荐模块作为独立能力旁挂于接口层，调用链为：
`app/api/ai/recommend/route.ts` → `app/lib/openai.ts` → OpenAI 兼容 API → 流式 SSE 回包至前端。

3.2 技术栈版本清单

分类	库 / 框架	版本	用途
全栈	Next.js	16.1.6	App Router + API Routes + SSR/SSG
全栈	TypeScript	5.x	类型安全，前后端类型共享
后端	Prisma	6.19.2	ORM + 自动迁移 + 类型生成
后端	jsonwebtoken	9.0.3	JWT 签发与校验
后端	bcryptjs	3.0.3	密码哈希（注册 salt rounds=10）
后端	Zod	4.3.6	结构化输出 Schema + 参数运行时校验
后端	LangChain + OpenAI	1.2.18 + 1.2.5	AI 推荐，支持 DouBao / Kimi 等兼容接口
后端	next-swagger-doc	0.4.1	JSDoc 注释自动生成 OpenAPI 文档
Web 前端	React	19.2.3	UI 框架
Web 前端	Ant Design	6.2.2	中后台组件库（Table/Drawer/Modal/Form/Tabs）
Web 前端	Recharts	3.7.0	LineChart / BarChart / PieChart 数据可视化
Web 前端	next-themes	0.4.6	深浅色主题持久化（`ThemeProvider`）

分类	库 / 框架	版本	用途
Web 前端	Axios	1.13.4	HTTP 客户端，封装于 `app/lib/request.ts`
小程序	Taro	4.1.11	跨平台编译（React 语法）
小程序	React	18.0.0	Taro 内部使用的 React 版本
小程序	dayjs	1.11.19	日历日期计算与格式化
数据库	MySQL	8.x	关系型数据库，InnoDB 事务

3.3 项目文件结构

项目由两个子工程组成：`trip_front_end_project`（Next.js 全栈应用）与 `trip_front_end_taro`（Taro 小程序）。以下为完整目录树，每个关键文件均附说明：

```
trip_front_end_project/
├── app/                              # Next.js 应用目录
│   ├── api/                          # 后端 API Routes（每个 route.tsx 一个端点）
│   │   ├── auth/
│   │   │   ├── login/route.tsx       # POST 登录，签发双 Token
│   │   │   ├── register/route.tsx    # POST 注册，创建用户（邮件去重 + bcrypt）
│   │   │   └── refresh/route.tsx     # POST 刷新 Access Token
│   │   ├── hotels/
│   │   │   ├── route.tsx             # GET 列表（多条件筛选）/ POST 创建
│   │   │   └── [id]/
│   │   │       ├── route.tsx         # GET 详情 / PUT 更新（含审核+AuditLog）/ DELETE
│   │   │       ├── room-types/route.tsx   # GET 该酒店房型列表
│   │   │       ├── bookings/route.tsx     # GET 该酒店预订列表
│   │   │       ├── review/route.tsx       # GET 该酒店评价列表
│   │   │       └── audit_log/route.tsx    # GET 审核日志
│   │   ├── bookings/
│   │   │   ├── route.tsx             # GET 预订列表 / POST 创建（事务）
│   │   │   └── [id]/route.tsx        # GET 详情 / PUT 状态更新
│   │   ├── room-types/
│   │   │   ├── route.tsx             # GET / POST 房型
│   │   │   └── [id]/
│   │   │       ├── route.tsx         # GET / PUT / DELETE 房型
│   │   │       └── availability/route.tsx # GET / POST 日期库存（批量 upsert）
│   │   ├── reviews/
│   │   │   ├── route.tsx             # GET 我的评价 / POST 提交评价（三重校验）
│   │   │   └── [id]/route.tsx        # DELETE 删除评价
│   │   ├── favorites/
│   │   │   ├── route.tsx             # GET 收藏列表 / POST 添加收藏（upsert）
│   │   │   ├── [id]/route.tsx        # DELETE 取消收藏
│   │   │   └── check/[id]/route.tsx  # GET 检查是否已收藏
│   │   ├── users/
│   │   │   ├── route.tsx             # GET 用户列表
│   │   │   ├── profile/route.tsx     # GET 当前用户资料（verifyAuth 保护）
│   │   │   └── [id]/
│   │   │       ├── route.tsx         # GET / PUT 用户详情
│   │   │       ├── bookings/route.tsx    # GET 某用户预订
│   │   │       └── favorite/route.tsx    # GET 某用户收藏
│   │   ├── locations/
│   │   │   ├── route.tsx             # GET / POST 城市
│   │   │   └── [id]/route.tsx        # PUT / DELETE 城市
│   │   ├── tags/
│   │   │   ├── route.tsx             # GET / POST 标签
```

```
│   │   │   └── [id]/route.tsx        # PUT / DELETE 标签
│   │   ├── room-availability/[id]/route.tsx  # 单条库存 GET / PUT
│   │   ├── ai/recommend/route.ts     # POST AI 流式推荐（LangChain, SSE 回包）
│   │   ├── upload/route.tsx          # POST 图片上传（文件名安全化）
│   │   ├── swagger/route.tsx         # GET Swagger JSON（JSDoc 自动生成）
│   │   └── utils/
│   │       ├── auth.ts               # verifyAuth() JWT 校验工具
│   │       └── permissions.ts        # Permission 名称检查工具
│   ├── admin/                        # 管理员页面（需 ADMIN 角色）
│   │   ├── layout.tsx                # 管理员侧边栏布局（AntD Layout）
│   │   ├── review/page.tsx           # 酒店审核页（Drawer 展示详情）
│   │   ├── hotels/page.tsx           # 酒店管理页（上下线操作）
│   │   ├── locations/page.tsx        # 城市管理页
│   │   └── tags/page.tsx             # 标签管理页
│   ├── merchant/                     # 商户页面（需 MERCHANT 角色）
│   │   ├── layout.tsx                # 商户侧边栏布局
│   │   ├── dashboard/page.tsx        # 数据看板（统计 + Recharts 图表）
│   │   ├── hotels/page.tsx           # 酒店管理（含房型子列表）
│   │   ├── bookings/page.tsx         # 预订管理（按状态筛选）
│   │   └── calendar/page.tsx         # 动态定价日历（批量 upsert）
│   ├── auth/
│   │   ├── login/page.tsx            # 登录页（深色/浅色模式自适应）
│   │   └── register/page.tsx         # 注册页（角色选择）
│   ├── components/
│   │   ├── AiChatWidget.tsx          # 悬浮 AI 聊天组件（ReadableStream 流式渲染）
│   │   ├── Navbar.tsx                # 导航栏（next-themes 主题切换）
│   │   ├── Footer.tsx                # 页脚
│   │   └── FeatureItem.tsx           # 首页特性介绍卡片
│   ├── services/                     # Web 前端 API 封装层（调用 request.ts）
│   │   ├── auth.ts                   # login / register / logout / getStoredUser
│   │   ├── hotel.ts                  # getHotels / createHotel / uploadImage ...
│   │   ├── booking.ts                # getMyBookings / createBooking ...
│   │   ├── room.ts                   # getRoomAvailability / updateAvailability
│   │   ├── review.ts                 # getMyReviews / createReview / deleteReview
│   │   └── admin.ts                  # CRUD locations / tags / approveHotel
│   ├── hooks/
│   │   └── useAuth.ts                # useAuth / useMerchantAuth / useAdminAuth
│   ├── lib/
│   │   ├── request.ts                # Axios 实例（拦截器 + 类型化方法）
│   │   ├── prisma.ts                 # PrismaClient 单例（防 Next.js 热更新泄漏）
│   │   ├── openai.ts                 # ChatOpenAI 实例配置（从 .env 读取）
│   │   └── swagger.ts                # Swagger 规范配置
│   ├── types/index.ts                # 全局 TypeScript 类型定义（User / Hotel / Booking...）
│   ├── providers.tsx                 # ThemeProvider（next-themes）等 Context 包装
│   ├── layout.tsx                    # 根布局（引入 providers / 全局字体）
│   └── page.tsx                      # 首页（登录/注册入口 + 系统介绍）
├── prisma/
│   ├── schema.prisma                 # 数据库模型定义（12 个 model）
│   ├── seed.ts                       # 基础种子数据（角色/权限/城市/标签）
│   └── seed-hotels.ts                # 酒店与房型示例数据
├── public/
│   └── uploads/                      # 图片上传存储目录
└── trip_front_end_taro/              # 小程序端（独立子项目）
    ├── src/
```

```
│   ├── pages/
│   │   ├── home/index.jsx          # 首页（搜索/日历/Banner/AI 入口）
│   │   ├── hotelList/index.jsx     # 酒店列表（接收 URL 搜索参数）
│   │   ├── hotelDetail/index.jsx   # 酒店详情（房型/预订/收藏/评分）
│   │   ├── hotelMap/index.jsx      # 地图页（双标记 + 导航按钮）
│   │   ├── orderList/index.jsx     # 订单列表（按状态分 Tab）
│   │   ├── orderDetail/index.jsx   # 订单详情（含总价/入住人信息）
│   │   ├── favoriteList/index.jsx  # 收藏列表
│   │   ├── reviewList/index.jsx    # 我的评价（提交/删除）
│   │   ├── login/index.jsx         # 登录页
│   │   ├── register/index.jsx      # 注册页（角色选择）
│   │   └── mine/index.jsx          # 个人中心（主题切换/退出）
│   ├── components/
│   │   ├── AiChatWidget/index.jsx      # 悬浮 AI 聊天（流式 chunk 渲染）
│   │   ├── BookingConfirm/index.jsx    # 预订确认弹窗（填写入住人）
│   │   ├── Calendar/index.jsx          # 日历组件（区间/单日模式）
│   │   ├── SearchSuggestion/index.jsx  # 搜索建议（历史+热门城市）
│   │   ├── FilterPanel/index.jsx       # 筛选面板（价格/星级）
│   │   ├── EmptyState/index.jsx        # 空状态占位组件
│   │   └── LoadingSpinner/index.jsx    # 加载态
│   ├── services/
│   │   ├── request.js            # Taro.request 封装（Token 注入/401 处理）
│   │   ├── hotel.js              # 酒店相关 API
│   │   ├── booking.js            # 预订相关 API
│   │   ├── favorite.js           # 收藏相关 API
│   │   ├── review.js             # 评价相关 API
│   │   └── auth.js               # 认证相关 API
│   ├── utils/
│   │   ├── useTheme.js           # 深色模式 Hook + eventCenter 广播
│   │   ├── format.js             # 日期/价格格式化工具
│   │   ├── storage.js            # Taro 本地存储封装
│   │   ├── constants.js          # 全局常量（API 地址等）
│   │   └── images.js             # Banner 图片配置
│   └── app.config.js             # Taro 页面路由/TabBar/权限声明
└── config/index.ts               # 多平台构建配置
```

3.4 模块划分

后端按业务域划分，每个模块对应 `app/api/` 下的独立路由组。模块间通过 Prisma Client 共享数据库连接，通过 `verifyAuth()` 共享鉴权工具，业务逻辑互不耦合。

模块	路由前缀	核心职责
认证模块	`/api/auth`	注册（邮件去重 + bcrypt 哈希）、登录（jwt.sign 双 Token）、Token 刷新
用户模块	`/api/users`	用户 CRUD、个人资料查询（`/profile`）、用户收藏与预订子路由
酒店模块	`/api/hotels`	多条件筛选列表、创建（嵌套 roomTypes + tagIds）、详情/更新/删除；状态变更在事务内写 AuditLog
房型模块	`/api/room-types`	房型 CRUD（校验 hotelId 归属）、按日期范围查询与批量 upsert RoomAvailability
预订模块	`/api/bookings`	事务性创建（逐日 upsert RoomAvailability.booked）、状态流转更新（pending→confirmed→…）
评价模块	`/api/reviews`	创建（验证 booking 归属 + 状态 + bookingId 唯一性）、列表查询、删除
收藏模块	`/api/favorites`	upsert 添加收藏、删除、列表查询、存在性检查（`/check/[id]`）
城市模块	`/api/locations`	城市 CRUD（仅管理员），供酒店创建时选择归属地
标签模块	`/api/tags`	标签 CRUD（仅管理员），供酒店多对多关联筛选

模块	路由前缀	核心职责
AI 推荐	`/api/ai/recommend`	LangChain 结构化意图提取（temperature=0）→ DB 候选筛选 → 流式 SSE 回包
文件上传	`/api/upload`	接收 multipart、文件名安全化（防路径穿越）、写入 `public/uploads/`
接口文档	`/api/swagger`	解析 JSDoc 注解，生成 OpenAPI JSON；`/api-doc` 页面渲染 Swagger UI

3.5 数据库设计

3.5.1 E-R 关系图

数据库共 12 个模型，核心实体及关系如下图所示。Location → Hotel 为 1:N，Hotel → RoomType → RoomAvailability 为 1:N:N；User 与 Hotel 通过 Booking 关联，Booking → Review 为 1:0..1（一单最多一评）；User 与 Hotel 通过 Favorite 多对多关联（复合主键）。

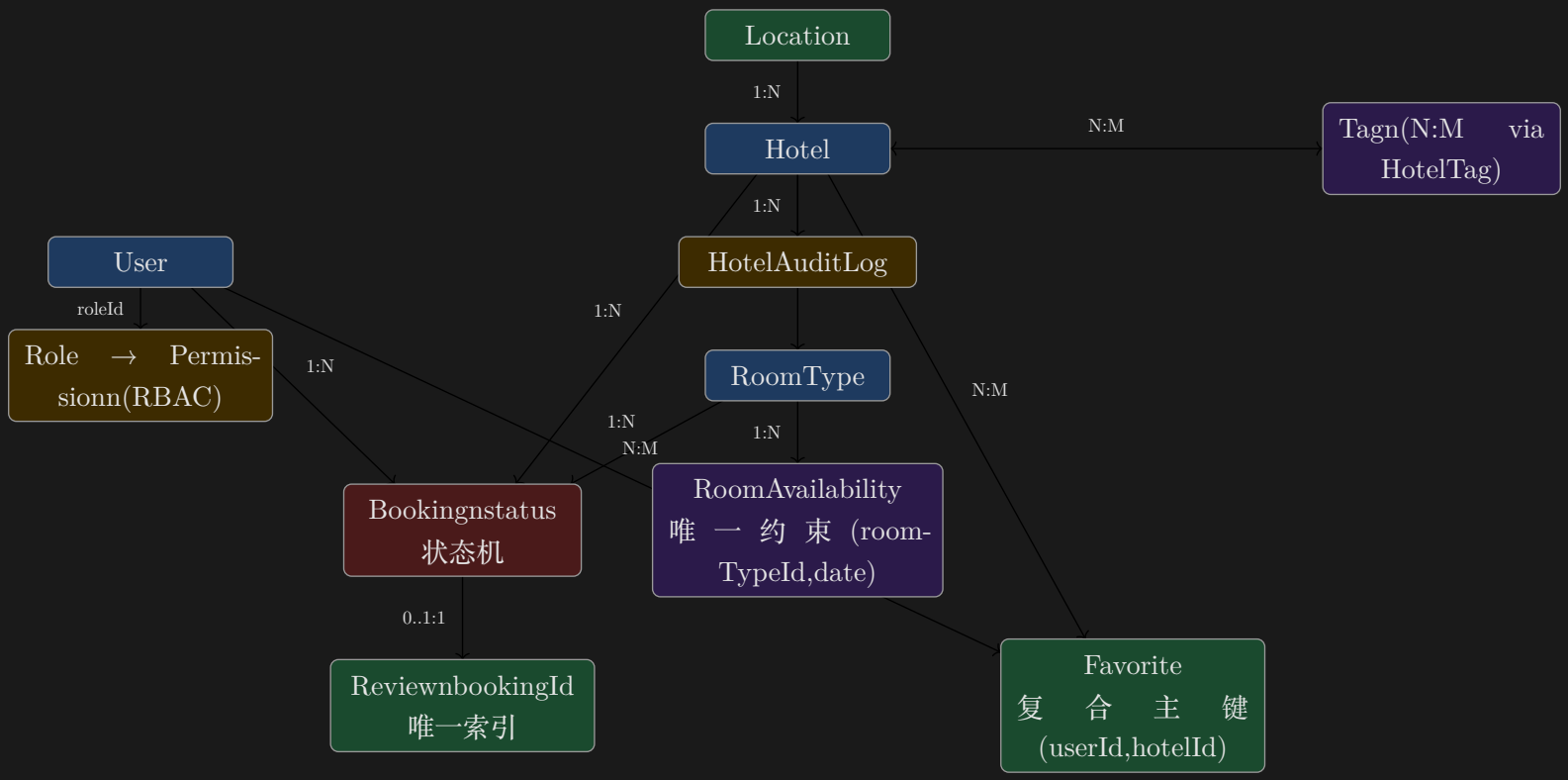

Figure 2: 易宿数据库 E-R 关系图

3.5.2 表结构详情

3.5.2.1 User（用户）

字段	类型	约束	说明
`id`	`Int`	PK, autoincrement	
`email`	`String`	unique	登录邮箱，注册时校验唯一
`phone`	`String?`	nullable	手机号
`password`	`String`	required	bcryptjs 哈希（salt rounds=10），明文不落库
`name`	`String?`	nullable	昵称
`roleId`	`Int`	FK→Role	1=USER / 2=MERCHANT / 3=ADMIN
`createdAt`	`DateTime`	default(now())	

3.5.2.2 Role（角色）

系统预置三种角色（USER / MERCHANT / ADMIN），通过 `User.roleId` 外键关联，实现基于角色的访问控制：

字段	类型	约束	说明
id	Int	PK, autoincrement	
name	String	unique	角色名称：USER / MERCHANT / ADMIN
description	String?	nullable	角色描述
createdAt	DateTime	default(now())	

3.5.2.3 Permission（权限）

细粒度权限条目，当前系统包含 HOTEL_AUDIT（审核酒店状态）、HOTEL_DELETE（删除任意酒店）等权限：

字段	类型	约束	说明
id	Int	PK, autoincrement	
name	String	unique	权限名称，如 HOTEL_AUDIT / HOTEL_DELETE
description	String?	nullable	权限描述
createdAt	DateTime	default(now())	

3.5.2.4 RolePermission（角色权限中间表）

多对多关联 Role 与 Permission，复合主键防止重复授权；ADMIN 角色在 seed.ts 中被分配全部权限：

字段	类型	约束	说明
roleId	Int	复合主键, FK→Role	级联删除（角色删除时自动清理）
permissionId	Int	复合主键, FK→Permission	级联删除（权限删除时自动清理）

3.5.2.5 Hotel（酒店）

酒店是系统核心实体，status 字段驱动审核状态机，latitude/longitude 存储 GCJ-02 坐标供小程序地图使用：

字段	类型	约束	说明
id	Int	PK	
merchantId	Int	FK→User	所属商户用户 ID
locationId	Int?	FK→Location, nullable	所在城市
nameZh	String	required	中文名称
nameEn	String?	nullable	英文名称
address	String	required	详细地址
starRating	Int?	nullable	星级 1–5
description	String?	Text, nullable	描述
facilities	Json	default([])	设施列表（JSON 字符串数组）
openingYear	Int?	nullable	开业年份
images	Json	default([])	图片 URL 数组，第一张为封面
latitude	Float?	nullable	纬度（GCJ-02 国测局坐标）
longitude	Float?	nullable	经度（GCJ-02 国测局坐标）
status	String	default("pending")	pending / published / rejected / offline
rejectionReason	String?	nullable	拒绝原因（审核拒绝时填写）
createdAt	DateTime	default(now())	
updatedAt	DateTime	updatedAt	Prisma 自动更新

3.5.2.6 HotelAuditLog（审核日志）

字段	类型	约束	说明
id	Int	PK	
hotelId	Int	FK→Hotel	被操作酒店

字段	类型	约束	说明
operatorId	Int	FK→User	操作人（通常为管理员）
oldStatus	String	required	变更前状态
newStatus	String	required	变更后状态
comment	String?	nullable	备注，如拒绝原因
createdAt	DateTime	default(now())	操作时间，用于审计追溯

此表在 `PUT /api/hotels/[id]` 发生状态变更时，于同一 `prisma.$transaction` 内原子写入，保证「状态变更」与「日志记录」要么同时成功要么同时回滚。

3.5.2.7 RoomType（房型）

字段	类型	约束	说明
id	Int	PK	
hotelId	Int	FK→Hotel	所属酒店
name	String	required	房型名称（如“豪华大床房”）
description	String?	nullable	
price	Decimal(10,2)	required	基础价格（元/晚），动态价格优先于此值
discount	Decimal(3,2)	default(1.00)	折扣系数 0.00–1.00
amenities	Json	default([])	房内设施 JSON 数组
images	Json	default([])	房型图片 URL 数组
stock	Int	default(10)	基础总库存，初始化 RoomAvailability.quota 的默认值
createdAt	DateTime	default(now())	

3.5.2.8 RoomAvailability（日期库存）

这是预订并发安全的核心表，每条记录代表某房型某日的库存快照：

字段	类型	约束	说明
id	Int	PK	
roomTypeId	Int	FK→RoomType	
date	DateTime	required	日期（精确到天，UTC 0 点）
price	Decimal(10,2)?	nullable	当日动态价格；为 null 时回退到 RoomType.price
quota	Int	required	当日可售间数（商户可通过日历配置）
booked	Int	default(0)	已预订间数，预订时事务内 `increment: 1` 原子递增
isClosed	Boolean	default(false)	为 true 时禁止预订（商户手动关闭房态）
createdAt	DateTime	default(now())	
updatedAt	DateTime	updatedAt	

唯一约束：`@@unique([roomTypeId, date])`，保证同一房型同一天只有一条记录，配合 `upsert` 实现幂等写入。

3.5.2.9 Booking（预订）

字段	类型	约束	说明
id	Int	PK	
userId	Int	FK→User	预订人
hotelId	Int	FK→Hotel	目标酒店
roomTypeId	Int	FK→RoomType	选择的房型
checkInDate	DateTime	required	入住日期
checkOutDate	DateTime	required	退房日期
guestCount	Int	default(1)	入住人数

字段	类型	约束	说明
totalPrice	Decimal(10,2)	required	总价（各日 RoomAvailability.price 之和）
status	String	default("pending")	pending→confirmed→checked_in→checked_out→completed / cancelled
guestInfo	Json?	nullable	入住人信息（姓名/手机等），格式自由 JSON
createdAt	DateTime	default(now())	

3.5.2.10 Review（评价）

字段	类型	约束	说明
id	Int	PK	
userId	Int	FK→User	评价人
hotelId	Int	FK→Hotel	被评酒店
bookingId	Int	唯一索引	关联订单；唯一索引防止一单多评
rating	Int	required	星级 1–5 整数
content	String	Text, required	文字评价
createdAt	DateTime	default(now())	
updatedAt	DateTime	updatedAt	

3.5.2.11 Favorite（收藏）

字段	类型	约束	说明
userId	Int	复合主键	收藏用户
hotelId	Int	复合主键	被收藏酒店
createdAt	DateTime	default(now())	

复合主键 `@@id([userId, hotelId])` 在数据库层面防止重复收藏，上层 `upsert` 操作保证幂等——同一用户对同一酒店的多次收藏请求只产生一条记录。

3.5.2.12 Location（城市）

存储酒店归属城市，由管理员维护；Hotel.locationId 外键关联：

字段	类型	约束	说明
id	Int	PK, autoincrement	
name	String	required	城市名称（如"北京"、"上海"）
description	String?	nullable	城市描述
createdAt	DateTime	default(now())	

3.5.2.13 Tag（标签）

描述酒店特色的标签（如"含早餐"、"免费 WiFi"），由管理员维护，通过 HotelTag 中间表与 Hotel 多对多关联：

字段	类型	约束	说明
id	Int	PK, autoincrement	
name	String	unique	标签名称（全局唯一，防止重复）
createdAt	DateTime	default(now())	

3.5.2.14 HotelTag（酒店标签中间表）

实现 Hotel 与 Tag 的多对多关联，复合主键保证同一酒店不会重复关联同一标签：

字段	类型	约束	说明
hotelId	Int	复合主键, FK→Hotel	级联删除（酒店删除时关联清理）
tagId	Int	复合主键, FK→Tag	

3.6 核心业务流程

3.6.1 用户登录认证流程

登录流程涉及 `app/api/auth/login/route.tsx`、`app/api/utils/auth.ts`、前端 `app/services/auth.ts` 和 `app/lib/request.ts`。系统采用双 Token 机制，Access Token 有效期 1 小时用于接口认证，Refresh Token 有效期 7 天用于无感刷新。

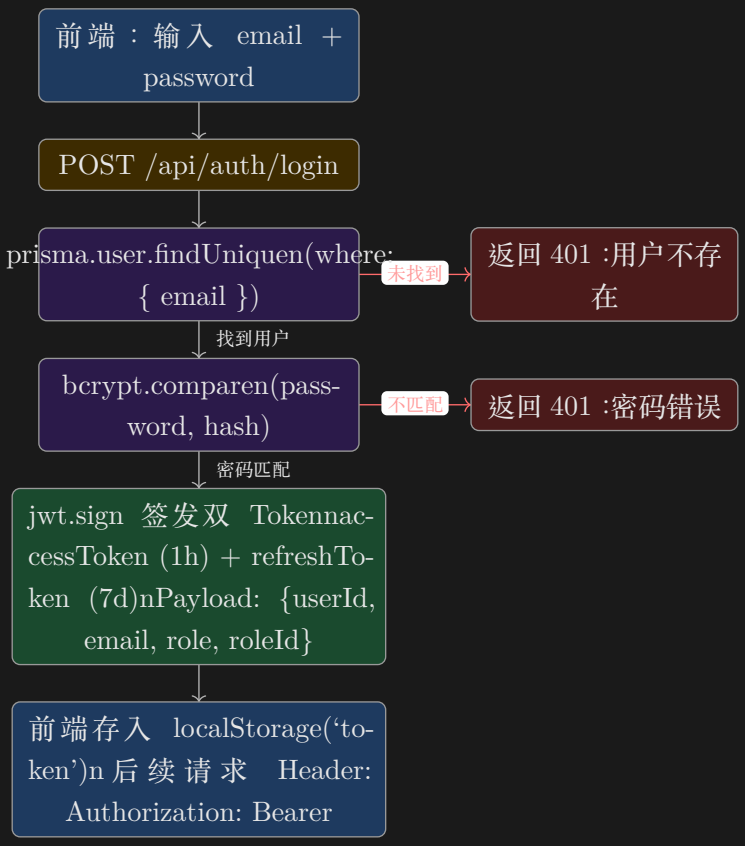

Figure 3: 用户登录认证流程

3.6.2 API 鉴权中间件

每个受保护的 API Route 在业务逻辑前统一调用 `verifyAuth(request)`（`app/api/utils/auth.ts`）。该函数提取 `Authorization` Header 中的 Bearer Token，使用 `jwt.verify()` 解码并返回 `DecodedUser`，失败时返回含 HTTP 状态码的错误对象。调用方直接解构结果进行早返回，代码模式统一整洁。

对于需要精细权限的操作（如审核通过/拒绝、删除酒店），在 JWT 角色校验通过后，还需额外查询 `role.rolePermission` 列表，检查是否包含 `HOTEL_AUDIT` 或 `HOTEL_DELETE` 权限名称。

3.6.3 在线预订事务流程

预订流程是系统最复杂的业务链路，涉及文件：小程序 `BookingConfirm/index.jsx` → `services/booking.js` → `app/api/bookings/route.tsx`。核心挑战是在高并发场景下防止超卖，通过数据库事务解决：

Figure 4: 在线预订事务流程

3.6.4 酒店审核状态机

酒店生命周期由 `status` 字段驱动，共 4 个状态，转换由 `PUT /api/hotels/[id]` 处理。状态转换时，`HotelAuditLog` 在同一事务内写入，保证操作可追溯。

Figure 5: 酒店审核状态机（每次状态变更均写入 HotelAuditLog）

3.6.5 AI 推荐流程

AI 推荐模块采用「意图提取 + 数据库筛选 + 流式生成」三阶段架构，涉及文件：小程序 `AiChatWidget/index.jsx` → `app/api/ai/recommend/route.ts` → `app/lib/openai.ts`。两次 LLM 调用使用不同温度参数：意图提取用 temperature=0 保证确定性，对话生成用 temperature=0.7 增加回复多样性。

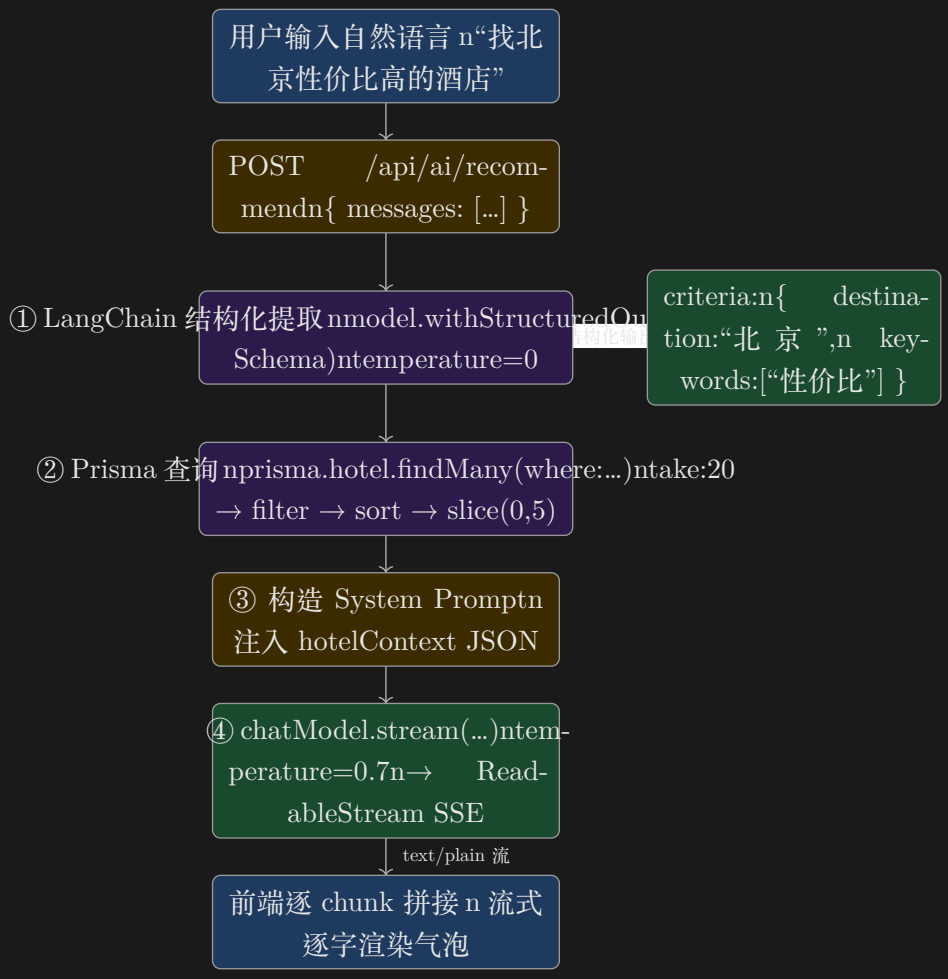

Figure 6: AI 推荐三阶段流程

4 具体实现

4.1 认证与鉴权实现

4.1.1 服务端鉴权 app/api/utils/auth.ts

该文件导出 `verifyAuth()` 函数，是系统所有受保护接口的统一鉴权入口。

做了什么：接收 HTTP 请求对象，提取 `Authorization` Header 中的 Bearer Token，调用 `jsonwebtoken` 的 `jwt.verify()` 完成签名验证与过期检查，解码得到 `{ userId, email, role, roleId }` 用户信息。

怎么做的：函数以值返回而非抛异常的方式表达结果——验证成功时返回 `{ success: true, user: DecodedUser }`，失败时返回 `{ success: false, error, status }`。调用方只需一个 `if (!authResult.success)` 判断即可完成错误提前返回与类型收窄，所有 route.tsx 文件头部均复用此模式，零重复。

4.1.2 前端路由保护 app/hooks/useAuth.ts

该文件导出三个 Hook：`useAuth(requiredRole?)`、`useMerchantAuth()`、`useAdminAuth()`。

做了什么：在 React 组件挂载后异步检查登录态与角色，未通过则自动跳转，通过则向调用方暴露当前用户对象，实现页面级访问控制。

怎么做的：检查逻辑放在 `useEffect` 中（声明 `'use client'`），不阻塞 SSR 首屏渲染。内部三重校验顺序执行：① `isAuthenticated()` 检查 `localStorage` 中是否存在 Token；② `getStoredUser()` 反序列化用户 JSON，为空则说明存储损坏；③ `currentUser.role?.name?.toUpperCase()` 与传入的 `requiredRole?.toUpperCase()` 做大小写不敏感比对。未登录跳 `/auth/login`，角色不匹配跳 `/`。校验通过后 `setUser(currentUser)` 并 `setLoading(false)`，页面根据 `loading` 渲染骨架屏，根据 `user` 决定是否展示敏感内容。`useMerchantAuth()` 和 `useAdminAuth()` 是对 `useAuth('merchant')` 和 `useAuth('admin')` 的语义化封装，减少调用方的字符串拼写错误。

4.1.3 权限精细化校验 app/api/hotels/[id]/route.tsx（PUT 方法）

做了什么：在 JWT 角色校验之上，对酒店状态变更操作额外验证细粒度权限，区分“商户下线自己的酒店”与“管理员审核/拒绝酒店”两种场景。

怎么做的：通过 Prisma 关联查询当前用户的完整权限列表（`role → rolePermission → permission`），检查是否含 `HOTEL_AUDIT` 或 `HOTEL_DELETE` 权限名称。下线操作允许本人或有 `HOTEL_AUDIT` 权限者执行；审核通过/拒绝仅限 `HOTEL_AUDIT` 权限。状态变更与 `HotelAuditLog` 写入在同一 `prisma.$transaction` 内原子完成，保证日志不漏不重。

4.2 预订事务实现

4.2.1 相关文件

- 小程序端入口：`trip_front_end_taro/src/components/BookingConfirm/index.jsx`（用户确认界面）
- 小程序服务层：`trip_front_end_taro/src/services/booking.js`（发起 HTTP 请求）
- 后端接口：`app/api/bookings/route.tsx`（POST 方法，核心事务逻辑）
- 数据库模型：`prisma/schema.prisma`（`RoomAvailability` 唯一约束定义）

4.2.2 做了什么

用户在 `BookingConfirm` 组件填写入住人信息并确认后，系统在数据库事务内完成库存校验、库存占用与订单创建三件事，保证高并发下不超卖。

4.2.3 怎么做的

后端 POST 接口先调用 `verifyAuth()` 确认用户已登录，再校验日期合法性，然后开启 `prisma.$transaction`。事务内对入住日期到退房日期之间的每一天依次执行：查询 `RoomAvailability` 记录 → 判断是否关闭（`isClosed`）或已售罄（`booked >= quota`）→ 如有异常立即抛错触发整体回滚 → 否则 `upsert` 将 `booked` 原子加一，同时累加当日价格到总价。全部日期通过后创建 `Booking` 记录（状态为 `pending`）。MySQL InnoDB 行锁在事务内保护同一日期的并发写操作互斥，任意步骤失败则所有 `booked` 变更一并回滚，从数据库层彻底消除超卖。

4.3 AI 推荐实现

4.3.1 相关文件

- 小程序悬浮入口：`trip_front_end_taro/src/components/AiChatWidget/index.jsx`（流式渲染 UI）
- Web 端悬浮入口：`app/components/AiChatWidget.tsx`（同功能）
- 后端接口：`app/api/ai/recommend/route.ts`（两阶段 LLM 调用）
- 模型配置：`app/lib/openai.ts`（ChatOpenAI 实例，从 `.env` 读取 API Key 和 Base URL）

4.3.2 做了什么

用户以自然语言描述需求（如"找北京带早餐性价比高的酒店"），系统自动理解意图、查询匹配酒店，并以流式打字机效果逐字返回推荐内容，支持多轮对话。

4.3.3 怎么做的

后端接口分两个阶段处理：

第一阶段——结构化意图提取：使用 LangChain 的 `model.withStructuredOutput(ZodSchema)` 以温度 0（确定性输出）调用模型，从最近 3 条对话消息中提取结构化搜索条件（目的地、价格区间、关键词、设施、排序方式）。Zod Schema 同时约束输出结构并与 TypeScript 类型同步。

第二阶段——流式推荐生成：用提取出的条件通过 Prisma 查询数据库，最多取 20 条已发布酒店，按价格/评分排序后取前 5 条，将酒店信息注入 System Prompt。再以温度 0.7（增加回复多样性）调用模型，将 LangChain 返回的 AsyncIterable 逐 chunk 写入 Web 标准 `ReadableStream`，以 `text/plain` 流式响应返回。前端 `AiChatWidget` 通过 `response.body.getReader()` 循环读取每个文字片段，实时拼接到当前消息气泡，实现逐字打印效果，无需额外 SSE 库。

前端 AiChatWidget 两端实现对比：

Web 端（`app/components/AiChatWidget.tsx`）基于 Ant Design `<FloatButton>` 固定定位于页面右下角，聊天窗口以 `<Card>` 浮层呈现。消息类型为 `{ role: 'user' | 'assistant', content: string }`，通过 `fetch` 接收流式响应——先向 `messages` 数组追加一条 `content: ''` 的占位 assistant 消息，再在 `while(true)` 循环内用 `TextDecoder` 解码每个 `Uint8Array` chunk 并追加到最后一条消息，每次更新触发 React 重渲染实现逐字效果。`Enter` 键直接发送（`Shift+Enter` 换行）。自动滚动通过 `useRef` 绑定末尾 `<div>` 并调用 `scrollIntoView({ behavior: 'smooth' })`。

小程序端（`AiChatWidget/index.jsx`）受微信小程序环境限制，`Taro.request` 不支持流式读取，因此退化为单次请求模式——发送后等待完整响应再渲染。悬浮按钮通过 `position: fixed` + `onTouchStart/Move/End` 三个触摸事件实现可拖拽效果：以 3px 位移阈值区分"点击"与"拖动"，拖动时计算新坐标并用 `Math.min/max` 限制在屏幕边界内，未发生移动的 `TouchEnd` 触发打开聊天窗口。消息列表用 `<ScrollView scrollIntoView={lastMsgId}>` 自动滚动至最新消息。

4.4 前端 HTTP 客户端实现

4.4.1 相关文件

- Web 端封装：`app/lib/request.ts`（Axios 实例 + 拦截器）
- 小程序端封装：`trip_front_end_taro/src/services/request.js`（Taro.request 封装）
- Web 业务服务层：`app/services/` 下的 `auth.ts`、`hotel.ts`、`booking.ts`、`room.ts`、`review.ts`、`admin.ts`
- 小程序业务服务层：`trip_front_end_taro/src/services/` 下各 `.js` 文件

4.4.2 做了什么

两端各自封装一个 HTTP 客户端单例，所有业务 Service 函数只需调用 `get/post/put/del` 四个类型化方法，Token 注入、响应解包、401 跳转均由客户端统一处理，业务代码零冗余。

4.4.3 怎么做的

Web 端（`request.ts`）：以 `axios.create({ baseURL: '/api', timeout: 10000 })` 创建实例，挂载两个拦截器。请求拦截器在 `typeof window !== 'undefined'` 守卫下从 `localStorage` 读取 Token，写入 `Authorization: Bearer <token>` Header，保证 SSR 环境不报错。响应拦截器的成功回调直接 `return response.data`，让调

用方无需每次解包；失败回调判断 status === 401 且 URL 不包含 /auth/login（避免登录页自身的错误密码 401 触发死循环），满足条件则清空 localStorage 并 window.location.href = '/auth/login' 硬跳转。最终导出泛型化的 get<T>/post<T>/put<T>/del<T> 方法，调用方可直接标注返回类型。

小程序端（request.js）：镜像相同结构，用 Taro.getStorageSync('token') 同步读取 Token，通过 Taro.request 发起请求，统一超时 10 秒。响应处理分三条路径：statusCode === 200 直接返回 res.data；statusCode === 401 清除存储并 Taro.navigateTo({ url: '/pages/login/index' })；其他错误码从 res.data.error 提取错误信息并以 Taro.showToast 展示给用户。网络错误（errMsg 包含 request:fail）和超时（包含 timeout）分别弹出对应提示。GET 请求额外实现了 Query 参数序列化：过滤 null/undefined 值后 encodeURIComponent 编码拼接为查询字符串。

4.5 商户数据看板实现

4.5.1 相关文件

- 页面组件：app/merchant/dashboard/page.tsx（数据聚合 + 图表渲染）
- 数据来源：app/services/booking.ts（getMyBookings 拉取商户全量预订）
- 图表库：Recharts 3.7.0（LineChart、BarChart、PieChart）

4.5.2 做了什么

商户进入看板页面后，自动展示今日预订数、本月营收、入住率三个 KPI 指标，以及近 7 天营收折线图、各月预订柱状图和房型分布饼图。

4.5.3 怎么做的

页面加载时调用 getMyBookings 一次性拉取当前商户的全量预订数据，随后在客户端纯 JavaScript 计算各项指标：今日预订通过入住日期与当天比对筛选；月营收对状态为 completed 或 checked_out 且本月创建的预订累加 totalPrice；7 天趋势数据按日期分桶聚合；房型分布将预订按 roomType.name 分组计数。所有聚合结果直接作为 Recharts 各图表的 data prop 传入，组件声明式渲染为 SVG 图表。此方案无需额外统计接口，但预订量极大时可考虑改为服务端聚合。

4.6 地图导航实现

4.6.1 相关文件

- 地图页面：trip_front_end_taro/src/pages/hotelMap/index.jsx
- 酒店详情入口：trip_front_end_taro/src/pages/hotelDetail/index.jsx（“在地图上查看”按钮）
- 数据来源：Hotel 表 latitude / longitude 字段（GCJ-02 国测局坐标系）

4.6.2 做了什么

用户在酒店详情页点击“地图”按钮后，跳转至 hotelMap 页面：同时显示酒店位置（绿色 callout 标记，含酒店名称）和用户当前定位（蓝色标记），并提供“导航到此”按钮，点击后调起微信内置地图导航。

4.6.3 怎么做的

hotelMap 页面使用 Taro 封装的 <Map> 组件，接收从路由参数传入的酒店 latitude/longitude/name。组件挂载时调用 Taro.getLocation({ type: 'gcj02' }) 获取用户当前坐标，将酒店和用户位置分别添加到 markers 数组（通过 iconPath、callout、width/height 控制样式）。“导航”按钮触发 Taro.openLocation({ latitude, longitude, name, address }) 直接拉起微信内置地图应用进行步行/驾车导航，无需集成第三方地图 SDK。坐标系统一使用 GCJ-02（高德/微信标准），与数据库存储的坐标格式一致，无需二次转换。

4.7 深色模式实现

4.7.1 相关文件

- 小程序主题 Hook：trip_front_end_taro/src/utils/useTheme.js
- 小程序主题工具函数：trip_front_end_taro/src/utils/theme.js（resolveTheme / getThemeCssVars / applyNativeTheme）
- 小程序使用方：各页面组件（home、hotelDetail、mine 等）根 View 绑定 style={cssVars}
- Web 端：app/providers.tsx（next-themes ThemeProvider）、app/components/Navbar.tsx（切换按钮）

4.7.2 做了什么

小程序端和 Web 端均支持深色/浅色主题切换，且多页面间状态实时同步。小程序通过 CSS 变量注入方式实现全局主题切换，无需给每个子组件单独传 props。

4.7.3 怎么做的

小程序端：`useTheme()` Hook 以懒初始化方式（`useState(() => ...)` 避免重复计算）读取 `resolveTheme()` 的结果——该函数优先读 `Taro.getStorageSync('theme')` 中的用户偏好，缺失时回退到 `Taro.getSystemInfoSync().theme` 系统设置。Hook 返回两个值：`cssVars`（一段 CSS 变量字符串，如 `--color-bg:#1a1a1a;--color-text:#fff;...`）和 `isDark` 布尔值。各页面将 `cssVars` 绑定到根 `<View style={cssVars}>`，子组件通过 `var(--color-xxx)` 继承，无需任何改动。

主题同步通过两条路径保证：① Taro 的 `useDidShow` 生命周期钩子在每次页面重新进入前台时重新计算并应用主题（处理从“我的”页切换主题后返回其他页的场景）；② `useEffect` 内监听 `Taro.eventCenter.on('themeChanged', handler)`，当用户在“我的”页手动切换时，`app.js` 广播该事件，所有已挂载页面的 Handler 同步更新状态，退出时 `off` 清理防止内存泄漏。

Web 端：由 `next-themes` 的 `ThemeProvider` 统一管理，`data-theme` 属性写入 `<html>` 标签，`globals.css` 通过 `[data-theme='dark']` 选择器覆写 CSS 变量。Navbar 调用 `useTheme().setTheme()` 完成切换，无需额外状态管理。

4.8 小程序首页搜索实现

4.8.1 相关文件

- 首页：`trip_front_end_taro/src/pages/home/index.jsx`
- 搜索建议组件：`trip_front_end_taro/src/components/SearchSuggestion/index.jsx`
- 日历组件：`trip_front_end_taro/src/components/Calendar/index.jsx`
- 数据接口：`trip_front_end_taro/src/services/hotel.js`（`getLocations`、`getTags`）

4.8.2 做了什么

用户进入首页后可选择住宿类型（国内 / 海外 / 钟点房 / 民宿）、目标城市、入离日期和筛选条件（价格区间、星级、设施标签），点击搜索跳转至酒店列表。首页同时提供搜索历史（最多保留 10 条，去重）、热门城市快捷入口、今晚 / 明天 / 周末等日期快捷选项，以及凌晨 0–6 时的早班提示。

4.8.3 怎么做的

页面挂载时并发请求 `getLocations()` 和 `getTags()` 获取城市与标签数据；搜索历史通过 `Taro.getStorageSync` 持久化，每次搜索前执行去重后追加，超过上限时删除最旧一条。日历组件根据当前 Tab 切换区间模式（普通住宿）与单日模式（钟点房）。点击搜索时将城市 ID、关键词、日期、价格区间、星级等参数序列化为 URL Query，经 `Taro.navigateTo` 传入酒店列表页，实现跨页面参数传递。

4.9 酒店列表与筛选实现

4.9.1 相关文件

- 列表页：`trip_front_end_taro/src/pages/hotelList/index.jsx`
- 筛选面板：`trip_front_end_taro/src/components/FilterPanel/index.jsx`
- 数据接口：`trip_front_end_taro/src/services/hotel.js`（`getHotels`）

4.9.2 做了什么

列表页展示符合搜索条件的酒店卡片，支持按推荐 / 价格升降序排序，以及价格区间 / 星级 / 设施多维度筛选；可一键切换为地图视图，在地图上以价格气泡标注各酒店位置，点击气泡显示酒店缩略卡片。

4.9.3 怎么做的

页面加载时将首页传入的 URL 参数解码为 `searchParams` 对象，调用 `getHotels()` 获取全量结果后存入 `hotelList`。所有后续的排序、筛选、关键词二次搜索均在客户端 `filterAndSortHotels()` 函数内完成，避免重复请求：关键词对酒店名称 / 描述 / 标签做大小写不敏感模糊匹配；价格区间和星级做精确过滤；设施标签要求全部命中（AND 逻辑）。`FilterPanel` 组件以受控方式接收 `defaultFilters` 并通过 `onConfirm` 回调将

新筛选值传回列表页触发重算。地图模式下将酒店坐标转换为 Taro <Map> 的 markers 数组，callout.content 显示价格，点击标记更新 selectedHotel 展示详情卡。

4.10 酒店详情页实现

4.10.1 相关文件

- 详情页：trip_front_end_taro/src/pages/hotelDetail/index.jsx
- 预订确认弹窗：trip_front_end_taro/src/components/BookingConfirm/index.jsx
- 数据接口：trip_front_end_taro/src/services/hotel.js、booking.js、favorite.js

4.10.2 做了什么

详情页展示酒店完整信息（图片画廊、星级、设施图标、评分与评价数），列出各房型的实时价格和剩余间数，用户选择房型与入离日期后点击预订，弹出确认弹窗填写入住人信息并提交订单。页面还提供收藏按钮（心形图标）和“在地图上查看”入口。

4.10.3 怎么做的

页面挂载时并发请求 getHotelById（酒店基本信息）和 getHotelRoomTypes(hotelId, startDate, endDate)（含该日期段动态价格与剩余间数）；若用户已登录则同时调用 checkFavorite 初始化收藏状态。滚动事件监听 scrollTop 在 0–150px 区间线性插值计算 Header 透明度，实现渐变浮现效果。房型卡片优先展示 dynamicPrice，回退至 RoomType.price，总价 = 日均价 × 入住晚数，实时显示在底部粘性栏。收藏操作捕获“已收藏”错误（idempotent upsert 语义），保证重复点击不报错。预订确认弹窗接收酒店、房型、日期、价格数据，用户填写入住人姓名 / 手机后调用 createBooking 提交，成功后跳转订单列表。

4.11 管理员酒店审核实现

4.11.1 相关文件

- 审核页面：app/admin/review/page.tsx
- 后端接口：app/api/hotels/[id]/route.tsx（PUT，状态变更 + AuditLog 原子写入）
- 权限控制：app/api/utils/permissions.ts（HOTEL_AUDIT 权限检查）

4.11.2 做了什么

管理员进入审核页后看到所有酒店的状态表格（待审核 / 已发布 / 已拒绝 / 已下线），可对每家酒店执行：查看详情（侧边 Drawer 展示图片、基本信息、房型列表）、通过审核、拒绝（填写拒绝理由）、下线、恢复上线。所有操作均有二次确认弹窗防止误操作。

4.11.3 怎么做的

表格以 Ant Design <Table> 渲染，通过 columns 的 render 函数根据酒店当前 status 字段动态展示不同操作按钮（pending 显示通过/拒绝；published 显示下线；offline 显示恢复）。点击“查看”将目标酒店写入 selectedHotel 并打开 <Drawer>，Drawer 内用 <Descriptions> 瀑布布局展示详情，<Image.PreviewGroup> 支持图片放大预览。拒绝操作弹出 <Modal> 要求填写拒绝理由，提交后调用 rejectHotel(id, reason) 将 status 设为 rejected 并附带理由；服务端在同一 prisma.$transaction 内同步写入 HotelAuditLog，保证审核记录不丢失。审核通过调用 approveHotel(id) 将状态改为 published，酒店立即出现在用户端搜索结果中。

5 接口文档

5.1 接口约定

约定项	说明
Base URL	`http://localhost:3000/api`（开发环境）
认证方式	HTTP Header：`Authorization: Bearer <accessToken>`
成功响应	`{ "success": true, "data": {...} }` 或 `{ "success": true, "data": [...] }`
错误响应	`{ "success": false, "error": "<描述>" }`，HTTP 状态码 400 / 401 / 403 / 404 / 500
Token 规则	Access Token 有效期 1h；过期后以 Refresh Token（7d）调用 `/auth/refresh` 换取新 Token，无感刷新
权限标注	`公开`=无需登录；`AUTH`=需登录（任意角色）；`MERCHANT`=商户角色；`ADMIN`=管理员角色
Swagger 文档	`http://localhost:3000/api-doc`（基于 JSDoc 注释自动生成 OpenAPI 3.0 规范）
日期格式	请求体中日期字段均使用 ISO 8601 字符串（`YYYY-MM-DD` 或 `YYYY-MM-DDTHH:mm:ssZ`）

5.2 认证接口 `/api/auth`

方法	路径	权限	请求体字段	响应说明
POST	`/auth/login`	公开	`email`（必填） `password`（必填）	返回 `{ accessToken, refreshToken, user: {id,email,name,role} }`；用户不存在或密码错误返回 401

POST	`/auth/register`	公开	`email`（必填）
			`password`（必填）
			`name?`
			`phone?`
			`role?`（“user”|“merchant”，默认“user”）
				返回 `{ success, data: User }`；邮箱已存在返回 400；密码经 bcrypt salt=10 哈希后存储

POST	`/auth/refresh`	公开	`refreshToken`（必填）	返回新 `accessToken`；Refresh Token 过期/无效返回 401

5.3 用户接口 `/api/users`

方法	路径	权限	参数	说明
GET	`/users`	ADMIN	—	所有用户列表，含角色信息
GET	`/users/profile`	AUTH	—	当前登录用户资料，包含 `id/email/name/phone/role` 等字段；由 `verifyAuth()` 保护，解析 Token 中 userId 查询
GET	`/users/[id]`	AUTH	—	指定用户资料详情
PUT	`/users/[id]`	AUTH	Body: `name? phone?` 等可选字段	更新用户信息，需为本人或 ADMIN
GET	`/users/[id]/bookings`	AUTH	—	该用户全量预订记录（管理员查询用）
GET	`/users/[id]/favorite`	AUTH	—	该用户收藏酒店列表

5.4 酒店接口 `/api/hotels`

方法	路径	权限	参数	说明
GET	`/hotels`	公开	Query: `locationId? status?`（默认 published）`tags?`（逗号分隔 ID，AND 匹配）`keyword?`（模糊匹配名称/地址）`merchantId?`	返回含 location / hotelTags / roomTypes 的酒店数组；tags 参数要求酒店同时拥有所有指定标签
POST	`/hotels`	MERCHANT	Body: `nameZh`、`address`、`merchantId`（必填）；`locationId?`、`starRating?`、	创建酒店，status 默认 pending，等待管理员审核才可被用户搜索到

方法	路径	权限	参数	说明
			`images?`、`facilities?`、`roomTypes[]?`、`tagIds[]?`	
GET	`/hotels/[id]`	公开	—	酒店详情，含 roomTypes / location / merchant({id,name,email}) / hotelTags
PUT	`/hotels/[id]`	本人或 ADMIN	Body: 任意 Hotel 字段；`status?`（见权限说明）；`rejectionReason?`	status 变更需 HOTEL_AUDIT 权限（offline 允许本人）；变更时事务内原子写 HotelAuditLog
DELETE	`/hotels/[id]`	本人或 ADMIN	—	本人所有者或拥有 HOTEL_DELETE 权限方可删除，删除酒店及关联数据
GET	`/hotels/[id]/room-types`	公开	—	该酒店所有房型列表
GET	`/hotels/[id]/bookings`	MERCHANT	—	该酒店全量预订列表，含 user / roomType 信息
GET	`/hotels/[id]/review`	公开	—	该酒店评价列表，含用户昵称/头像信息
GET	`/hotels/[id]/audit_log`	ADMIN	—	该酒店状态变更历史，含 operator 信息和操作时间

5.5 预订接口 `/api/bookings`

`POST /api/bookings` 是系统最复杂的接口，内部执行数据库事务（详见第 4.2 节）。任何日期售罄或关闭时返回 500 并携带具体错误信息，客户端应提示用户重新选择日期。

方法	路径	权限	参数	说明
GET	`/bookings`	AUTH	Query: `merchantId?`（商户查名下全部预订，必须等于当前用户 ID）	用户查本人预订（含 hotel / roomType 信息）；merchantId 鉴权防止枚举他人订单
POST	`/bookings`	AUTH	Body: `hotelId`、`roomTypeId`、`checkInDate`、`checkOutDate`（必填）；`guestCount?`（默认 1）、`guestInfo?`（JSON）	事务创建；售罄/关闭返回 500（含日期信息）；成功返回 201 + Booking 记录
GET	`/bookings/[id]`	AUTH	—	预订详情，含 hotel / roomType / user 信息
PUT	`/bookings/[id]`	AUTH	Body: `status`（新状态）	状态流转：pending→confirmed→checked_in→checked_out / completed / cancelled；各状态由商户或用户按业务触发

5.6 房型与库存接口 `/api/room-types`

方法	路径	权限	参数	说明
GET	`/room-types`	公开	Query: `hotelId?`	房型列表，可按酒店过滤
POST	`/room-types`	MERCHANT	Body: `hotelId`、`name`、`price`（必填）；`description?`、`stock?`（默认 10）、`discount?`（默认 1.00）、`amenities?`、`images?`	创建房型，内部验证 hotelId 属于当前商户，防止越权
GET	`/room-types/[id]`	公开	—	房型详情
PUT	`/room-types/[id]`	MERCHANT	Body: 任意房型字段	更新房型，验证归属关系
DELETE	`/room-types/[id]`	MERCHANT	—	删除房型，验证归属关系

方法	路径	权限	参数	说明
GET	`/room-types/[id]/availability`	公开	Query: `startDate`、`endDate`	按日期范围返回 RoomAvailability 列表，前端日历组件用于渲染价格/库存
POST	`/room-types/[id]/availability`	MERCHANT	Body: `data[]`（每项含 `date`、`price?`、`quota?`、`isClosed?`）	批量 upsert 日期库存与动态价格，商户日历批量设价使用此接口

5.7 评价接口 `/api/reviews`

方法	路径	权限	参数	说明
GET	`/reviews`	AUTH	—	当前用户所有评价，含 hotel（名称/图片）/ booking（日期）信息
POST	`/reviews`	AUTH	Body: `bookingId`、`rating`（1–5 整数）、`content`（必填）	三重校验：① booking.userId === 当前用户，② booking.status 为 completed 或 checked_out，③ bookingId 在 Review 表唯一（防一单多评）
DELETE	`/reviews/[id]`	AUTH	—	仅评价本人可删除，其他用户返回 403

5.8 收藏接口 `/api/favorites`

方法	路径	权限	参数	说明
GET	`/favorites`	AUTH	—	收藏列表含酒店完整详情；返回的 id 字段格式为 `"userId-hotelId"`（便于前端直接用作 React key）
POST	`/favorites`	AUTH	Body: `hotelId`（必填）	upsert 写入，数据库复合主键保证幂等，重复收藏不报错
DELETE	`/favorites/[id]`	AUTH	Path: id = hotelId	按 hotelId 取消当前用户的收藏
GET	`/favorites/check/[id]`	AUTH	Path: id = hotelId	返回 `{ isFavorited: boolean }`，详情页初始化心形按钮状态使用

5.9 AI 推荐接口 `POST /api/ai/recommend`

权限：公开（无需登录，降低使用门槛）

请求体格式：发送完整对话历史，支持多轮上下文理解。

```
{
  "messages": [
    { "role": "user",      "content": "找北京带早餐的五星级酒店" },
    { "role": "assistant", "content": "为您推荐以下酒店..." },
    { "role": "user",      "content": "有没有价格更低的？" }
  ]
}
```

响应格式：`Content-Type: text/plain; charset=utf-8`（流式文本），前端通过 `ReadableStream.getReader()` 逐块读取文字片段拼接渲染。

内部处理步骤：① LangChain 结构化提取意图（最近 3 条消息，temperature=0）→ ② Prisma 查询并过滤候选酒店（最多取 20 条，客户端排序后取前 5）→ ③ 构造含酒店上下文的 System Prompt → ④ ChatOpenAI 流式生成返回（temperature=0.7）。

5.10 文件上传接口 `POST /api/upload`

权限：AUTH（需登录，防止未授权用户滥用存储）

请求：`Content-Type: multipart/form-data`，字段名 `file`

文件名安全化处理：过滤原始文件名中的非字母数字字符，拼接时间戳与随机串，防路径穿越攻击：

`${Date.now()}-${Math.random().toString(36).slice(2)}-${sanitized_original_name}`

响应：

```
{ "success": true, "url": "/uploads/1704067200000-abc123-hotel.jpg" }
```

返回的 `url` 直接可用于 `<img src>` 或存入 Hotel.images JSON 数组。生产环境建议将 `public/uploads/` 替换为 OSS/CDN 存储，此接口仅需修改写入目标路径。

6 部署与运行

6.1 环境依赖

依赖	推荐版本	说明
Node.js	≥ 20.x LTS	Next.js 16 要求 Node 18+，推荐 20 LTS 以获得最佳兼容性
npm	≥ 10.x	随 Node.js 附带安装
MySQL	8.x	本地安装或 Docker；需提前创建数据库 `hotel_management`
微信开发者工具	≥ 1.06.x	调试与预览小程序；需使用微信开发者账号登录
OpenAI 兼容接口	—	AI 推荐功能必须；支持 DouBao（火山引擎）、Kimi（月之暗面）、OpenAI 官方等兼容 `/v1/chat/completions` 的接口

6.2 环境变量配置

在项目根目录创建 `.env` 文件（此文件包含密钥，**严禁提交至版本控制**，已在 `.gitignore` 中排除）：

```
# ── 数据库连接字符串 ──
DATABASE_URL="mysql://root:yourpassword@localhost:3306/hotel_management"

# ── JWT 签名密钥（建议使用 32 位以上随机字符串，生产环境必须更换）──
JWT_SECRET="your-super-secret-jwt-key-here"

# ── AI 大模型配置（以火山引擎 DouBao 为例）──
OPENAI_API_KEY="your-api-key"
BASE_URL="https://ark.cn-beijing.volces.com/api/v3"
MODEL_ID="ep-20241231xxxxxx-xxxxx"

# 若使用 OpenAI 官方接口，BASE_URL 可省略，MODEL_ID 改为 "gpt-4o" 等
```

6.3 种子数据与测试账号

执行 `npm run db:seed` 后，系统自动写入以下基础数据（幂等，可重复执行）：

角色与权限：创建 USER / MERCHANT / ADMIN 三个角色，写入 `HOTEL_AUDIT`、`HOTEL_DELETE` 等细粒度权限，并将所有权限分配给 ADMIN 角色。

城市数据：预置 20 个热门城市（北京、上海、广州、深圳、杭州、成都等）。

标签数据：预置 20 个酒店标签（含早餐、免费 WiFi、游泳池、靠近地铁、温泉酒店等）。

示例酒店：执行 `prisma/seed-hotels.ts` 可额外写入多家含房型、图片、标签的示例酒店，供界面预览使用。

测试账号创建：种子脚本本身不写入用户数据，需通过注册接口或直接 SQL 插入创建测试账号。以下为推荐的测试账号结构：

角色	示例邮箱	示例密码	说明
ADMIN	`admin\@hotel.com`	admin123	注册后需在数据库将 roleId 改为 3（ADMIN）
MERCHANT	`merchant\@hotel.com`	merchant123	注册时选择“商户”角色，roleId=2
USER	`user\@hotel.com`	user123	默认注册即为普通用户，roleId=1

注意：bcryptjs 哈希密码需通过注册接口写入，不可明文存储。如需批量创建测试账号，可在 `prisma/seed.ts` 中追加 `prisma.user.create` 代码，密码使用 `bcryptjs.hashSync('明文密码', 10)` 预哈希。

6.4 后端 + Web 启动

```
# 1. 安装项目依赖
npm install

# 2. 同步数据库结构至 MySQL
```

```
#    首次运行或 schema.prisma 变更后必须执行
npx prisma db push

# 3. 生成 Prisma Client 类型文件
#    db push 通常自动触发，可手动补充执行
npx prisma generate

# 4. 写入种子数据（角色/权限/城市/标签/示例酒店，可选但推荐）
npm run db:seed

# 5. 启动开发服务器（含热更新）
npm run dev
# 访问地址:
#   首页            → http://localhost:3000
#   管理员后台      → http://localhost:3000/admin/review
#   商户后台        → http://localhost:3000/merchant/dashboard
#   API 文档（Swagger）  → http://localhost:3000/api-doc

# ── 生产构建与启动 ──
npm run build
npm run start
```

6.5 小程序端启动

```
# 1. 进入小程序子项目目录
cd trip_front_end_taro

# 2. 安装依赖
npm install

# 3. 启动微信小程序编译（watch 模式，保存后自动重编译）
3
# 编译产物位于 trip_front_end_taro/dist/weapp/

# 4. 在微信开发者工具中导入项目：
#    文件 → 导入项目 → 选择 dist/weapp 目录
#    AppID 填写自己的小程序 AppID 或选择"测试号"

# ── 其他平台 ──
npm run dev:h5          # H5 浏览器预览
npm run dev:alipay      # 支付宝小程序
npm run build:weapp     # 生产构建（微信），输出到 dist/weapp/
```

注意：小程序端 API 基础地址在 `trip_front_end_taro/src/services/request.js` 顶部配置，开发环境默认 `http://localhost:3000/api`。真机调试时需改为局域网 IP（如 `http://192.168.1.x:3000/api`）或已备案的线上域名，同时在微信小程序管理后台的「开发设置 → request 合法域名」中添加对应域名。

6.6 常见问题排查

问题现象	排查与解决方案
`navigateBack:fail`	开发者工具直接打开非首页时的正常报错，从 TabBar 首页正常跳转或真机运行均不会出现，可忽略
`getLocation` 权限被拒	`app.config.js` 中已声明 `permission.scope.userLocation`；开发者工具需在右上角“详情”→“本地设置”中手动允许地理位置权限

问题现象	排查与解决方案
数据库连接失败	确认 MySQL 服务已启动（`brew services start mysql` 或 `systemctl start mysql`）；检查 `.env` 中 DATABASE_URL 的用户名/密码/库名；执行 `npx prisma db push` 可直接验证连通性
AI 接口返回 500	检查 `.env` 中 OPENAI_API_KEY / BASE_URL / MODEL_ID 是否正确；国内网络访问 OpenAI 官方接口需配置代理或改用 DouBao/Kimi 等国内兼容接口
图片上传后 404	确认 `public/uploads/` 目录存在且对 Node 进程有写权限；Next.js 开发模式下 public 目录变更需重启服务；生产环境建议改为 OSS 持久化存储
小程序地图显示空白	开发者工具：「设置 → 项目设置 → 不校验合法域名」；真机需在小程序后台添加 request 合法域名，并确认腾讯地图 SDK 使用了正确的 key
Prisma 类型报错	执行 `npx prisma generate` 重新生成 Client；确认 `prisma/schema.prisma` 已保存；确认 `.env` DATABASE_URL 指向正确数据库；生成文件路径在 `app/generated/prisma/`
端口 3000 被占用	`npm run dev -- -p 3001` 指定其他端口；同步修改小程序端 `services/request.js` 中的 baseURL；修改微信小程序后台 request 合法域名
Token 过期后请求 401	Web 端 `request.ts` 的 401 拦截器自动清除 localStorage 并跳转登录页；小程序端 `request.js` 同理跳转 login 页；如需无感刷新，使用 `/auth/refresh` 接口在 401 时换取新 Token